VIET NAM NATIONAL UNIVERSITY - HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

INTRODUCTION TO ARTIFICIAL INTELLIGENCE

REPORT PROJECT

FUTOSHIKI

GROUP 11: TURTLE

Theory Lecturer	Dr.	Bui Duy Dang
Lab Lecturer	M.Sc.	Le Nhut Nam
	B.Sc.	Nguyen Thanh Tinh
Students	24127163	Nguyen Son Hai
	24127004	Ngo Tien Binh
	24127242	Nguyen Hoang Phuc Thinh

Contents

1 Introduction **3**
1.1 Overview . . . 3
1.2 Objectives . . . 3
2 Problem formalization **4**
2.1 Logic Vocabulary . . . 4
2.2 First-Order Logic Axioms . . . 4
2.3 Conversion to Conjunctive Normal Form (CNF) . . . 5
3 Inference & Search algorithms **6**
3.1 Knowledge Base Grounding Engine . . . 6
3.2 Forward Chaining Implementation . . . 7
3.3 Backward Chaining Implementation . . . 9
3.4 A* Search Strategy . . . 11
3.5 Baseline Solver . . . 13
4 System Design & Implementation **16**
4.1 Environment . . . 16
4.2 Input/Output Data Format . . . 16
5 Experimental Results & Analysis **19**
5.1 Test Case Descriptions . . . 19
5.2 Performance Comparison Tables . . . 20
5.3 Comparative Analysis . . . 21
6 Conclusion & future work **24**

Information Team

0.1 Member Information

Student ID	Name	Email	Role
24127163	Nguyen Son Hai	24127163@student.hcmus.edu.vn	Leader
24127004	Ngo Tien Binh	24127004@student.hcmus.edu.vn	Member
24127242	Nguyen Hoang Phuc Thinh	24127242@student.hcmus.edu.vn	Member

Table 1. List of member

0.2 Task Allocation Table

Member	Assigned Tasks	Completion
Nguyen Son Hai	- Logical modeling (FOL + CNF) - Knowledge base construction and problem representation - Report writing - Management of project and progress	100%
Ngo Tien Binh	- Data processing system - Implement search algorithms (A* & backtracking) - Experiments and result collection (time, memory, steps) - Demonstration video recording	100%
Nguyen Hoang Phuc Thinh	- Implement inference algorithms (forward & backward chaining) - Performance comparison and analysis - Demonstration video recording	100%

Table 2. Task Assignment and Completion Status of Group Members

Note: All members actively participated in discussions, code reviews, and the completion of the final report. The task assignments listed above represent primary responsibilities only and do not indicate exclusive contributions.

0.3 Mandatory requirements

No.	Requirement	Status	Remarks
1	Formal FOL Axioms Intelligence algorithms	Completed	Defined all core axioms and manually derived CNF forms for non-trivial rules
2	Automatic KB Generation	Completed	Implemented a grounding engine to generate Propositional/CNF theories for any N
3	Forward Chaining	Completed	Fact propagation and contradiction detection from scratch.
4	Backward Chaining	Completed	SLD Resolution (Prolog-style) to query cell values.
5	A Guided Search*	Completed	A* algorithm with an Admissible Heuristic (e.g., Remaining Cells).
6	Baseline Algorithm	Completed	Standard Backtracking solver for performance comparison.
7	Input/Output Handling	Completed	Testing on 10 cases (4×4 to 9×9) with comparative charts.
8	Experimental Report	Completed	Conducted a rigorous analysis with charts comparing time, memory, and inference steps.
9	Source code and README	Completed	GitHub repository
10	Demo video in drive	Completed	Video demo

Table 3. Self-Evaluation of Mandatory Requirements

1. Introduction

1.1. Overview

Futoshiki is a logic-based puzzle that originated in Japan. It is played on an N×N grid where the player must fill in numbers from 1 to N. The rules are quite similar to Sudoku regarding row and column uniqueness, but the core challenge of Futoshiki lies in the inequality constraints ($<$ or $>$) placed between adjacent cells (Puzzle Futoshiki, 2026). These signs force a specific numerical order that must be strictly followed to reach a valid solution.

In this project, we treat Futoshiki not just as a game, but as a formal logic problem. By using First-Order Logic (FOL), we can translate human rules into a language that a computer can process. We then apply and compare various AI techniques—from basic Backtracking to more advanced Inference Engines and A* Search—to find the most efficient way to solve puzzles of different sizes, ranging from 4×4 to 9×9.

1.2. Objectives

The main goals of this project are:

- **Modeling:** Translating the game's rules into formal FOL axioms and then converting them into Conjunctive Normal Form (CNF) for the computer to process.
- **Algorithm Implementation:** Developing several solvers including Forward Chaining, Backward Chaining, Backtracking, and A* Search.
- **Benchmarking:** Running tests on grids from 4×4 to 9×9 to compare which algorithm is the fastest and most memory-efficient.
- **Application:** Building a GUI using Flet that allows users to pick a test case and watch the solver work in real-time.

2. Problem formalization

In this project, we need to turn the rules of Futoshiki into a format the computer can solve. We use First-Order Logic (FOL) to define these rules and then convert them into Conjunctive Normal Form (CNF) clauses for our inference engines.

2.1. Logic Vocabulary

We define a set of predicates to represent the board state and constraints (based on problem statement and apply in KBGenerator.py):

- $Val(i, j, v)$: The cell at row i, column j is filled with value v.
- $Given(i, j, v)$: The cell (i, j) has a pre-filled number v.
- $LessH(i, j)$ / $GreaterH(i, j)$: There is a "<" or ">" sign between horizontal neighbors (i, j) and $(i, j + 1)$.
- $LessV(i, j)$ / $GreaterV(i, j)$: There is a "<" or ">" sign between vertical neighbors (i, j) and $(i + 1, j)$.
- $Less(v_1, v_2)$: This is a basic math check that returns True if v_1 is smaller than v_2.

2.2. First-Order Logic Axioms

We define the following rules to ensure the puzzle is solved correctly (Puzzle Futoshiki, 2026):

- A1 (Completeness): Every single cell must have at least one value from 1 to N.
  $\forall i \forall j \exists v (1 \leq v \leq N \wedge Val(i, j, v))$
- A2 (Cell Uniqueness): A cell cannot have two different values at once.
  $\forall i \forall j \forall v_1 \forall v_2 (Val(i, j, v_1) \wedge Val(i, j, v_2) \wedge v_1 \neq v_2 \Rightarrow \perp)$
- A3 (Row Uniqueness): Every number in a row must be unique.
  $\forall i \forall j_1 \forall j_2 \forall v (Val(i, j_1, v) \wedge Val(i, j_2, v) \wedge j_1 \neq j_2 \Rightarrow \perp)$
- A4 (Column Uniqueness): Every number in a column must be unique.
  $\forall j \forall i_1 \forall i_2 \forall v (Val(i_1, j, v) \wedge Val(i_2, j, v) \wedge i_1 \neq i_2 \Rightarrow \perp)$
- A5 (Horizontal Inequalities): This rule ensures the numbers follow the horizontal "<" or ">" signs.
  $\forall i \forall j \forall v_1 \forall v_2 (LessH(i, j) \wedge Val(i, j, v_1) \wedge Val(i, j + 1, v_2) \Rightarrow Less(v_1, v_2))$
  $\forall i \forall j \forall v_1 \forall v_2 (GreaterH(i, j) \wedge Val(i, j, v_1) \wedge Val(i, j + 1, v_2) \Rightarrow Less(v_2, v_1))$
- A6 (Vertical Inequalities): This rule ensures the numbers follow the vertical "<" or ">" signs.
  $\forall i \forall j \forall v_1 \forall v_2 (LessV(i, j) \wedge Val(i, j, v_1) \wedge Val(i + 1, j, v_2) \Rightarrow Less(v_1, v_2))$
  $\forall i \forall j \forall v_1 \forall v_2 (GreaterV(i, j) \wedge Val(i, j, v_1) \wedge Val(i + 1, j, v_2) \Rightarrow Less(v_2, v_1))$
- A7 (Respect Given Clues): Any cell that already has a number from the start must keep that value.
  $\forall i \forall j \forall v (Given(i, j, v) \Rightarrow Val(i, j, v))$

2.3. Conversion to Conjunctive Normal Form (CNF)

Below is how we convert each FOL rule into CNF clauses:

2.3.1. Cell Completeness (A1)

We turn the "exists" into a list of options for each cell. Step: Choose $v = 1$ OR $v = 2$ OR ... OR $v = N$.

CNF:

$$(Val_{i,j,1} \vee Val_{i,j,2} \vee \cdots \vee Val_{i,j,N})$$

2.3.2. Uniqueness (A2, A3, A4)

For any two things that cannot happen together (like two values in one cell), we use:

$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$$

Example A2:

$$(\neg Val(i, j, v_1) \vee \neg Val(i, j, v_2))$$

Example A3:

$$(\neg Val(i, j_1, v) \vee \neg Val(i, j_2, v))$$

Example A4:

$$(\neg Val(i_1, j, v) \vee \neg Val(i_2, j, v))$$

2.3.3. Inequality Constraints (A5, A6)

We use:

$$(P \wedge Q \wedge R \Rightarrow S) \equiv (\neg P \vee \neg Q \vee \neg R \vee S)$$

Since signs and comparisons are known facts, we only generate clauses for invalid assignments.

LessH example:

$$(\neg Val(i, j, v_1) \vee \neg Val(i, j + 1, v_2)) \quad \text{when } v_1 \geq v_2$$

GreaterH example:

$$(\neg Val(i, j, v_1) \vee \neg Val(i, j + 1, v_2)) \quad \text{when } v_1 \leq v_2$$

Vertical constraints (A6) follow the same idea.

2.3.4. Given Clues (A7)

If $Given(i, j, v)$ is true, then:

$$(Val(i, j, v))$$

This becomes a unit clause.

3. Inference & Search algorithms

3.1. Knowledge Base Grounding Engine

The grounding engine is responsible for turning our high-level FOL axioms into a massive list of propositional clauses (CNF) that the computer can actually solve. This process happens in our KBGenerator.py file.

3.1.1. Variable Mapping

In a logic problem, every possibility needs a unique ID. Since we have a 3D state space (row, column, and value), we use a mathematical formula to map each $(row, col, value)$ triplet into a single unique integer:

$$ID = (r \times N^2) + (c \times N) + v$$

where:

- r, c: the coordinates of the cell (0 to $N - 1$)
- v: the value assigned to that cell (1 to N)
- N: the size of the grid

We also have a decode function that does the reverse. It takes an integer ID and tells us exactly which row, column, and value it represents, which is how we print the final board.

3.1.2. Clause Generation

Our engine builds the Knowledge Base (KB) by looping through the grid and generating clauses for each rule we defined in Section 2:

- **Possibilities:** For every cell, it creates a clause saying the cell must be 1 OR 2 OR ... OR N.
- **Conflicts:** For row and column uniqueness, it generates pairs of negative literals. For example, "It cannot be that $(0, 0) = 1$ AND $(0, 1) = 1$."
- **Inequalities:** This is the most important part. If the puzzle has a "$<$" sign between two cells, the engine loops through all possible number combinations. If a combination like $5 < 3$ is mathematically impossible, it adds a clause to "block" that specific pair of values.

By the end of this process, the grounding engine produces a clean list of lists (clauses) that our Forward and Backward Chaining solvers use to reach a conclusion.

3.2. Forward Chaining Implementation

Forward Chaining is a "data-driven" inference method. It starts with the known facts (the clues) and applies logical rules to deduce new facts until the puzzle is solved or no more deductions can be made (Puzzle Futoshiki, 2026).

3.2.1. Initialization

First, the solver takes the pre-filled numbers from the board and adds them to a set called *facts*. Each starting number is treated as a "proven" truth.

3.2.2. The Inference Loop (Modus Ponens)

The core of the solver is a loop that repeatedly checks our list of CNF clauses. We use a simplified version of Modus Ponens:

For each clause, we look at the literals (the individual Val variables).

If every literal in a clause is proven to be False except for one, then that last literal must be True.

For example, if a clause says (Cell A is 1 OR Cell A is 2) and we already know Cell A is NOT 1, the solver concludes that Cell A is 2.

Once a new fact is found, it is added to the *facts* set, and the loop starts over.

3.2.3. Fixpoint and Contradictions

Fixpoint: The solver keeps looping until it reaches a "fixpoint"—a state where an entire pass through the rules produces no new facts.

Contradiction Check: During the process, if the solver ever proves that a cell must be a certain value, but another rule has already proven that it cannot be that value, a contradiction is detected. In this case, the solver stops and returns "Inconsistent."

Finally, the *_extract_solution* function reads all the positive facts from our set and fills them into the $N \times N$ grid to show the result.

3.2.4. Pseudo code

```
Procedure ForwardChaining(KB, clues)
    Facts = clues
    While True:
        NewFactsFound = False
        For each Clause in KB:
            If Clause is already satisfied by Facts:
                Continue
            UnknownLiterals = Literals in Clause not in Facts

            // Modus Ponens: If only one literal is unknown
            If count(UnknownLiterals) == 1:
                Literal L = UnknownLiterals[0]
                If -L in Facts:
                    Return "Inconsistent" // Contradiction
                Facts.add(L)
                NewFactsFound = True

        If NewFactsFound == False:
            Break // Reached Fixpoint
    Return ExtractSolution(Facts)
End Procedure
```

Implementation Details

In our ForwardChaining.py, the solver loops through all CNF clauses. If it finds a clause where all literals but one are proven False (negations are in our fact set), it infers that the last literal must be True. This process repeats until the board is complete or no more deductions are possible.

Illustrative Example (4×4 Trace):

Suppose we have a 4×4 board where $Cell(0,0) = 4$ is a given clue. There is a constraint $Cell(0,0) > Cell(0,1)$.

Initialization: Our Facts set starts with $\{Val(0,0,4)\}$.

Clause Selection: The solver looks at a clause generated for $GreaterH(0,0)$, which looks like:

$$(\neg Val(0,0,4) \vee Val(0,1,1) \vee Val(0,1,2) \vee Val(0,1,3))$$

Literal Elimination: The solver sees that $Val(0,0,4)$ is in Facts, so the literal $\neg Val(0,0,4)$ is False.

Unit Rule: If other rules (like column uniqueness) have already proven that $Cell(0,1)$ cannot be 1 or 2, then $Val(0,1,1)$ and $Val(0,1,2)$ are also False.

Deduction: Now, $Val(0,1,3)$ is the only unknown literal left in the clause. Following the pseudocode, count(UnknownLiterals) == 1 becomes true. The solver adds $Val(0,1,3)$ to the Facts set and sets NewFactsFound = True.

Loop: The loop starts again because new data was found, continuing until the entire board is derived.

3.3. Backward Chaining Implementation

Backward Chaining is a "goal-driven" approach. Instead of applying all rules to everything, it starts with a specific goal (like "What is the value of cell (0,0)?") and works backward to see if the known facts and rules support that goal (Puzzle Futoshiki, 2026).

3.3.1. Initialization and Unit Propagation

Before starting the heavy work, the solver performs a quick cleanup:

Fact Setup: It adds all starting clues to a *derived_facts* set.

Unit Propagation: It scans the rules for "unit clauses" (rules with only one possible answer). It solves these immediately to narrow down the search space before moving to the recursion.

3.3.2. Goal Selection with MRV

To decide which cell to solve next, the solver uses the Minimum Remaining Values (MRV) heuristic. Instead of picking cells in order, it looks for the cell that is most restricted (has the fewest possible numbers that could fit). By solving the "hardest" cells first, we reduce the chances of having to backtrack later.

3.3.3. SLD Resolution Logic

For each chosen cell, the solver tries values from 1 to N and attempts to prove them using a simplified SLD Resolution:

Check for Conflict: For a specific value, it checks every rule in the Knowledge Base.

Remainder Check: If a rule contains the negation of our value (meaning "this cell cannot be X"), the solver checks the rest of that rule. If the rest of the rule is already proven False, then our chosen value must also be False (a contradiction).

Recursion: If a value is proven to be a candidate, the solver temporarily assigns it and moves to the next goal. If it hits a dead-end, it "backtracks" by removing the assignment and trying the next number.

3.3.4. Pseudo code

```
Procedure BackwardChain(Board, Facts, KB)
    If all cells are filled:
        Return Board // Goal reached

    // Choose the most restricted cell
    Cell (r, c) = SelectCellWithMRV(Board)

    For each Value V from 1 to N:
        Fact = Encode(r, c, V)

        // SLD Resolution: Try to prove this value is consistent
        If CanProve(Fact, Facts, KB):
            Board[r, c] = V
            Facts.add(Fact)

            If BackwardChain(Board, Facts, KB) is Success:
                Return Board

            // Backtrack if it fails
            Board[r, c] = 0
            Facts.remove(Fact)

    Return Failure
End Procedure
```

Implementation Details

The core of this solver is the *_can_prove_fact* function, which acts as a simplified SLD Resolution engine. Before the main recursion, we run Unit Propagation to solve any cells that only have one possible legal value. During the search, we use the MRV (Minimum Remaining Values) heuristic to pick the next cell to solve. By focusing on cells with the fewest choices first, we prune the search tree significantly and find contradictions faster.

Illustrative Example (4×4 Trace):

Consider a board where the solver needs to fill $Cell(1, 1)$.

Cell Selection: SelectCellWithMRV checks all empty cells. $Cell(1, 1)$ has only two legal choices (2 or 3) due to row constraints, so it is picked first.

Trial 1: The loop starts with $V = 2$. The solver creates the goal $Fact = Val(1, 1, 2)$.

Verification: CanProve checks the KB. It finds a clause for column uniqueness:

$$(\neg Val(1, 1, 2) \vee \neg Val(0, 1, 2))$$

Since $Cell(0, 1)$ is already 2, the rule is violated. CanProve returns False.

Trial 2: The loop moves to $V = 3$. This time, CanProve finds no contradictions in the KB.

Recursion: The solver sets $Board[1, 1] = 3$, adds it to Facts, and calls BackwardChain again to solve the next cell.

Backtrack: If the following steps fail (Failure), the code executes $Board[1,1] = 0$ and Facts.remove, then returns to try another value.

3.4. A* Search Strategy

A* is an informed search algorithm that finds the solution by exploring the most promising states first. It uses a Priority Queue to manage board states based on the cost function:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$: The number of cells already filled (path cost).
- $h(n)$: The estimated cost to reach the goal (heuristic).

In our implementation, we don't just pick any empty cell. We use the Minimum Remaining Values (MRV) heuristic to pick the cell with the fewest legal numbers. If there is a tie, we use the Degree Heuristic to choose the cell involved in the most inequality constraints, helping to prune the search tree quickly.

3.4.1. Heuristic Function Design

For this project, we designed our heuristic as:

$$h(s) = \text{The number of empty cells remaining on the board.}$$

In our AStar.py code, the $h(board)$ function also acts as a pruning tool. While counting empty cells, it checks if any empty cell has zero possible legal values. If it finds such a cell, the function returns infinity (∞). This tells the solver that the current board state is a dead-end, so it immediately stops and tries a different path.

```
Function Heuristic(Board)
    EmptyCount = 0
    For each Cell (r, c) in Board:
        If Board[r, c] == 0:
            PossibleValues = GetLegalValues(Board, r, c)
            If count(PossibleValues) == 0:
                Return Infinity // Pruning: No legal moves left
            EmptyCount = EmptyCount + 1
    Return EmptyCount
End Function
```

Illustrative Example (4×4 Trace):

This function scans the board to estimate the remaining work.

- **Scanning:** It finds an empty cell at $(2, 2)$. GetPossibleValues checks row/column/inequality rules and finds that only $\{4\}$ is legal. EmptyCount becomes 1.
- **Scanning:** It moves to $Cell(3, 3)$. However, because of a "<" sign and existing numbers, GetPossibleValues returns an empty list [].
- **Pruning:** The code hits the condition If count(PossibleValues) == 0. It immediately returns Infinity.
- **Effect:** This tells the A* solver that this board is a "dead-end" and should never be explored, which is exactly how our code avoids wasting time on wrong paths.

3.4.2. Admissibility Proof

A heuristic $h(s)$ is admissible if it never overestimates the true cost to reach the goal ($h^*(s)$) (Puzzle Futoshiki, 2026).

Actual Cost ($h^*(s)$): In Futoshiki, we must fill every remaining empty cell exactly once. Therefore, the true number of steps to the goal is exactly the number of empty cells.

Our Heuristic ($h(s)$): We count the number of empty cells.

Proof: Since our estimate $h(s)$ is always exactly equal to the number of steps needed (and never more), it satisfies the condition $h(s) \leq h^*(s)$. This proves our heuristic is admissible, guaranteeing that A* will find a solution if it exists.

3.4.3. Pseudo code

```
Procedure AStarSearch(InitialBoard)
    // Priority Queue stores (f_value, g_value, board)
    OpenList = PriorityQueue()
    OpenList.push(h(InitialBoard), 0, InitialBoard)
    VisitedStates = Set()

    While OpenList is not empty:
        (f, g, CurrentBoard) = OpenList.pop()

        If CurrentBoard is full:
            Return CurrentBoard // Goal Reached

        // Choose best cell using MRV + Degree Heuristic
        (r, c) = SelectBestCell(CurrentBoard)

        For each Value V in PossibleValues(CurrentBoard, r, c):
            NewBoard = ApplyAssignment(CurrentBoard, r, c, V)
            If NewBoard not in VisitedStates:
                H_new = Heuristic(NewBoard)
                If H_new != Infinity:
                    OpenList.push(g + 1 + H_new, g + 1, NewBoard)
                    VisitedStates.add(NewBoard)
    Return Failure
End Procedure
```

Illustrative Example (4×4 Trace):

Suppose the OpenList (Priority Queue) has one board state where 10 cells are filled ($g = 10$).

Pop: The solver pops the state with the lowest f. Let's say $f = 10 + 6 = 16$.

Selection: SelectBestCell uses MRV and finds $Cell(3, 3)$ is the most constrained cell.

Expansion: The solver tries $V = 4$. It creates NewBoard with $Cell(3, 3) = 4$.

Heuristic Check: It calls Heuristic(NewBoard). Since 5 cells are now empty, $H_{new} = 5$.

Push: A new cost is calculated: $f = (10 + 1) + 5 = 16$. This NewBoard is pushed into the OpenList. The algorithm always prioritizes boards where h is smaller, meaning we are getting closer to the 16-cell goal.

3.5. Baseline Solver

To evaluate the performance of our logic-based solvers, we implemented a standard Backtracking algorithm as a baseline. This is a simple recursive Depth-First Search (DFS) that treats the puzzle as a basic Constraint Satisfaction Problem (CSP) without using complex logical inference (Puzzle Futoshiki, 2026).

3.5.1. How it Works

The solver in Backtracking.py follows a straightforward process:

Cell Selection: It moves through the board in a fixed order (top-to-bottom, left-to-right), looking for the first empty cell (value 0).

Trial and Error: For each empty cell, it attempts to assign numbers from 1 to N in sequence.

Validation: Every time a number is assigned, the solver calls *_check_valid*. This function manually checks the assigned value against three things:

- Row uniqueness.
- Column uniqueness.
- Inequality constraints ($<$, $>$) with neighbors that already have values.

Recursion and Backtracking: If the value is valid, it moves to the next cell. If it reaches a point where no numbers from 1 to N work, it "backtracks" by resetting the current cell to 0 and returning to the previous cell to try a different number.

This solver is called "baseline" because it does not use any heuristics (like MRV) or logical propagation (like Forward Chaining). It relies entirely on brute-force recursion, making it a perfect point of comparison for our more advanced AI algorithms.

3.5.2. Pseudocode

```
Procedure Backtrack(Board, row, col)
    // If we reached the end of the board, we are done
    If row == N:
        Return True

    // Calculate next cell position
    NextRow, NextCol = GetNextCell(row, col)

    // Skip if cell is already filled (clue)
    If Board[row, col] != 0:
        Return Backtrack(Board, NextRow, NextCol)

    // Try values 1 to N
    For each Value V from 1 to N:
        If IsValid(Board, row, col, V):
            Board[row, col] = V // Try assignment

            If Backtrack(Board, NextRow, NextCol) is True:
                Return True

            Board[row, col] = 0 // Undo (Backtrack)

    Return False
End Procedure
```

Implementation Details

In our Backtracking.py, the IsValid function manually checks three things every time a number is tried:

Uniqueness: Ensures the number doesn't already exist in the current row or column.

Inequalities: Checks the neighbors (top, bottom, left, right). If there is a "<" or ">" sign, it ensures the current value respects that math relation.

Simplicity: This solver is "blind" because it doesn't use any heuristics like MRV or any logical deduction. It simply tries every combination until it finds one that works, which makes it slower for larger 9×9 puzzles but useful as a comparison point.

Illustrative Example (4×4 Trace):

- **Fixed Path:** The solver starts at Index 0 ($Cell(0, 0)$). Since it's a clue, it skips to Index 1.
- **Trial:** At $Cell(0, 1)$, it tries $V = 1$. IsValid checks the row, column, and inequalities. If valid, it recurses to Index 2.
- **Failure:** At $Cell(0, 2)$, it tries $V = 1, 2, 3, 4$, but all fail IsValid.
- **Backtrack:** The function returns False. The code jumps back to $Cell(0, 1)$, resets it to 0, and the loop tries $V = 2$. It continues this until all 16 indices are successfully filled.

4. System Design & Implementation

4.1. Environment

Our Futoshiki solver is developed using Python 3.11 and is designed to be cross-platform (Windows, macOS, Linux). The system uses a modern stack to handle the logic, interface, and performance analysis:

- **Flet (v0.28.3):** Used to build the desktop GUI. It allows us to load test cases from a dropdown menu and visualize both the initial and solved grids.
- **Logic Processing:** Custom Python modules (`ForwardChaining.py`, `AStar.py`, etc.) implement the core AI algorithms.
- **Data & Visualization:** NumPy handles the grid arrays, while Matplotlib generates the benchmark charts (execution time, success rates, and heatmaps).
- **Requirements:** All dependencies are listed in `requirements.txt` and can be installed using:

```
pip install -r requirements.txt
```

4.2. Input/Output Data Format

The application uses a strict text-based format to communicate with the solver engines, ensuring all test cases are processed consistently.

4.2.1. Input Data Format (test01.txt)

The input files are structured into four distinct parts:

- **Grid Size (N):** The first line defines the board dimension.
- **Initial Grid:** N lines where values are separated by commas. 0 represents an empty cell, while $1..N$ are pre-filled clues.
- **Horizontal Constraints:** N lines, each containing $N-1$ values.
  - 1 represents a "Less than" ($<$) sign between (i, j) and $(i, j+1)$.
  - -1 represents a "Greater than" ($>$) sign.
  - 0 indicates no constraint.
- **Vertical Constraints:** $N-1$ lines, each containing N values, representing signs between rows.
  - 1 represents a "Less than" (top $<$ bottom) sign.
  - -1 represents a "Greater than" (top $>$ bottom) sign.

Example:

```
4
0,0,2,4
0,4,0,0
0,0,0,1
2,1,0,0
1,0,1
1,-1,1
-1,1,-1
-1,1,-1
1,1,-1,-1
1,-1,0,-1
-1,-1,1,1
```

Actual state:

```
. < .   2 < 4
^   ^   v   v
. < 4 > . < .
^   v       v
. > . < . > 1
v   v   ^   ^
2 > 1 < . > .
```

4.2.2. Output Data Format (output01.txt)

The solver generates a detailed report including:

- **Metadata:** Displays the filename and the grid size (e.g., `test01.txt | Size: 4x4`).
- **Initial Board Visualization:** A formatted view of the board using symbols: `.` for empty cells, numbers for clues, and arrows (`<`, `>`, `v`, `^`) for the constraints.
- **Solving Time:** The exact time in milliseconds (ms) taken by the chosen algorithm.
- **Final Result:** The fully completed grid, verified against all FOL axioms. If no solution is found, it reports `"Inconsistent."`

```
============================================================
 FILE: test01.txt   |   Kích thước: 4x4
============================================================

 [ Bảng ban đầu ]
. < .   2 < 4
^   ^   v   v
. < 4 > . < .
^   v       v
. > . < . > 1
v   v   ^   ^
2 > 1 < . > .

 Thời gian giải: 0.965 ms

 [ KẾT QUẢ: GIẢI HOÀN TOÀN ]
1 < 3   2 < 4
^   ^   v   v
3 < 4 > 1 < 2
^   v       v
4 > 2 < 3 > 1
v   v   ^   ^
2 > 1 < 4 > 3

============================================================
```

Figure 1. Output format

5. Experimental Results & Analysis

5.1. Test Case Descriptions

We used 10 test cases (test01.txt to test10.txt) to evaluate our solvers. These cases vary in grid size from 4×4 to 9×9, allowing us to observe how the complexity grows as the search space expands.

Test: test01.txt (Size: 4x4, Given: 6)		
✓ Forward Chaining	:	0.365 ms
✓ Backward Chaining	:	5.841 ms
✓ A* Search	:	0.750 ms
✓ Backtracking	:	0.049 ms
✓ Secret	:	0.169 ms

Test: test02.txt (Size: 5x5, Given: 9)		
✓ Forward Chaining	:	1.069 ms
✓ Backward Chaining	:	37.023 ms
✓ A* Search	:	1.118 ms
✓ Backtracking	:	0.092 ms
✓ Secret	:	0.236 ms

Test: test03.txt (Size: 6x6, Given: 10)		
× Forward Chaining	:	1.565 ms
✓ Backward Chaining	:	235.669 ms
✓ A* Search	:	5.118 ms
✓ Backtracking	:	0.196 ms
✓ Secret	:	0.252 ms

Test: test04.txt (Size: 7x7, Given: 14)		
× Forward Chaining	:	2.681 ms
✓ Backward Chaining	:	895.398 ms
✓ A* Search	:	20.903 ms
✓ Backtracking	:	1.284 ms
✓ Secret	:	0.444 ms

Test: test05.txt (Size: 8x8, Given: 16)		
× Forward Chaining	:	8.702 ms
✓ Backward Chaining	:	4291.669 ms
✓ A* Search	:	36.133 ms
✓ Backtracking	:	3.378 ms
✓ Secret	:	0.424 ms

Test: test06.txt (Size: 9x9, Given: 40)		
✓ Forward Chaining	:	9.369 ms
✓ Backward Chaining	:	3246.272 ms
✓ A* Search	:	13.092 ms
✓ Backtracking	:	0.709 ms
✓ Secret	:	0.737 ms

Test: test07.txt (Size: 4x4, Given: 5)		
× Forward Chaining	:	0.302 ms
✓ Backward Chaining	:	5.594 ms
✓ A* Search	:	1.167 ms
✓ Backtracking	:	0.035 ms
✓ Secret	:	0.112 ms

Test: test08.txt (Size: 5x5, Given: 5)		
× Forward Chaining	:	1.249 ms
✓ Backward Chaining	:	147.695 ms
✓ A* Search	:	5.292 ms
✓ Backtracking	:	0.944 ms
✓ Secret	:	0.396 ms

Test: test09.txt (Size: 6x6, Given: 9)		
× Forward Chaining	:	2.193 ms
✓ Backward Chaining	:	287.030 ms
✓ A* Search	:	7.319 ms
✓ Backtracking	:	0.523 ms
✓ Secret	:	0.232 ms

Test: test10.txt (Size: 7x7, Given: 11)		
× Forward Chaining	:	3.359 ms
✓ Backward Chaining	:	2910.549 ms
✓ A* Search	:	32.595 ms
✓ Backtracking	:	5.912 ms
✓ Secret	:	0.535 ms

5.2. Performance Comparison Tables

The following table summarizes the data collected from our `benchmark.py` script:

Algorithm	Min Time (ms)	Max Time (ms)	Avg Time (ms)	Success Rate
Forward Chaining	0.302	9.369	3.085	30%
Backward Chaining	5.594	4291.669	1206.274	100%
A* Search	0.750	36.133	12.349	100%
Backtracking	0.035	5.912	1.312	100%
Secret Solver	0.112	0.737	0.354	100%

5.3. Comparative Analysis

We generated several visualizations to compare the algorithms across different dimensions.

5.3.1. 1. Execution Time per Test Case

The overall performance for each specific test case is shown in the bar and line charts below.

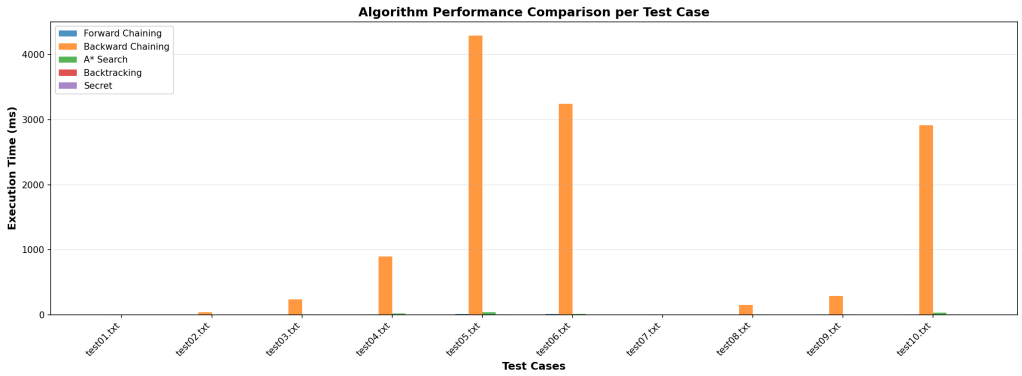

Figure 2. Algorithm Performance Comparison per Test Case.

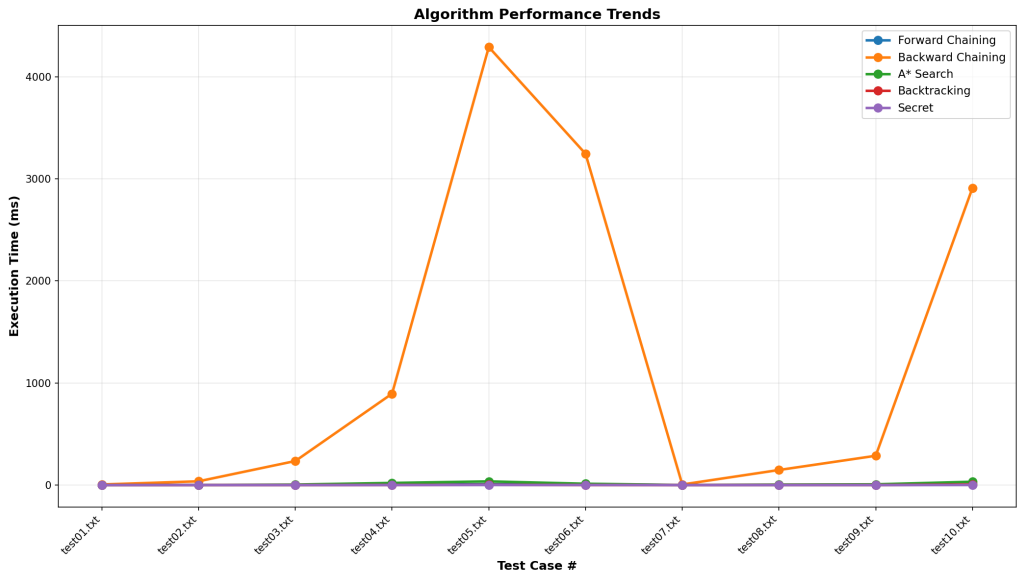

Figure 3. Algorithm Performance Trends across different test cases.

Looking at the Algorithm Performance Comparison (Figure 2) and Trends (Figure 3), there is a significant performance gap between the solvers.

Backward Chaining is the slowest method due to the overhead of processing a large number of CNF clauses for every inference step.

Backtracking and the Secret Solver achieve the best performance because they rely on direct constraint checking rather than expensive logical inference.

A* Search performs in the middle range; it is faster than Backward Chaining because its heuristic function reduces the search space, but it is still slower than Backtracking due to the additional overhead of maintaining and updating the priority queue during execution.

5.3.2. Performance Density and Heatmap

To get a clearer view of the "speed zones" of each solver, we look at the heatmap:

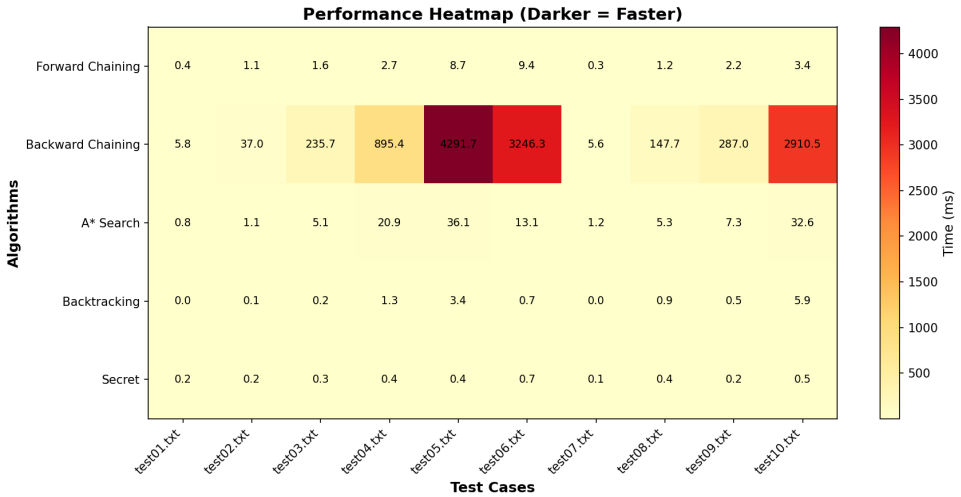

Figure 4. Performance Heatmap (Darker colors represent faster execution).

Forward Chaining has a low success rate of approximately 30%. It appears to be "fast" only because it terminates immediately when it cannot deduce a value using pure logical inference. As a result, it fails on most grids since it does not include any search or backtracking mechanism to handle uncertain or incomplete states.

In contrast, Backward Chaining, A* Search, and Backtracking all achieve a 100% success rate. Although they employ different strategies, their recursive and search-based nature allows them to explore alternative possibilities, ensuring that a valid solution is eventually found even in more complex 9×9 cases.

5.3.3. Statistical Distribution

The Boxplot helps us understand the stability and variance of each algorithm:

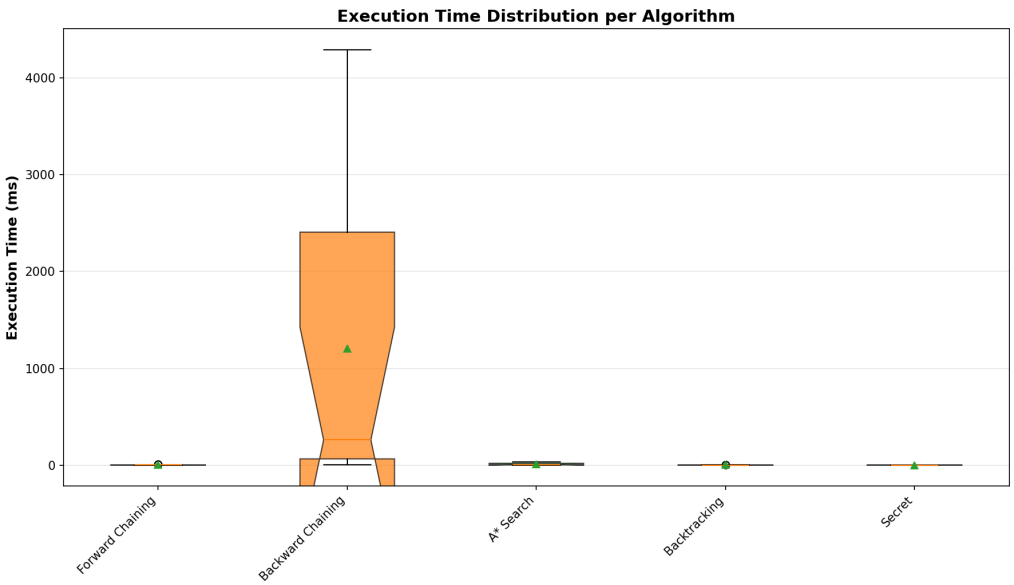

Figure 5. Execution Time Distribution (Boxplot).

Backward Chaining is the most unstable algorithm, with execution time fluctuating significantly from approximately 5 ms to over 4000 ms as N increases. This indicates poor scalability when applied to larger and more complex grids.

In contrast, Backtracking and A* Search demonstrate high stability. Their boxplots have very small spreads, meaning their execution times remain consistent and predictable regardless of puzzle difficulty.

The Secret Solver exhibits almost zero variance in execution time, making it the most stable and scalable approach for solving large-scale Futoshiki puzzles.

5.3.4. Final Average Ranking

The chart below shows the final average ranking of all algorithms:

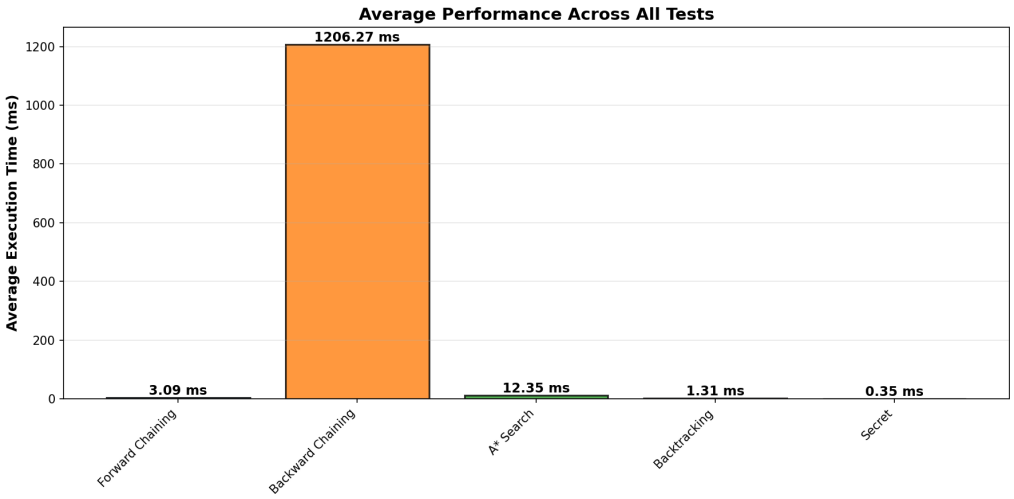

Figure 6. Average Performance Across All Tests.

Pure Logic approaches (Forward Chaining and Backward Chaining) are useful for simple deductions; however, they are either too slow (Backward Chaining) or incomplete (Forward Chaining) when applied to full game-solving scenarios.

Heuristic Search (A*) represents a more "intelligent" approach by guiding the search using an evaluation function. However, for a problem of this scale, both Optimized Constraint Propagation (Secret Solver) and Simple Backtracking are more practical in terms of efficiency.

The Secret Solver achieves the best benchmark performance (average 0.35 ms) because it effectively combines the advantages of both approaches: fast search and aggressive pruning of invalid or impossible assignments.

6. Conclusion & future work

In this project, we successfully modeled the Futoshiki puzzle as a Constraint Satisfaction Problem (CSP) using First-Order Logic. We implemented and compared four main AI strategies: Forward Chaining, Backward Chaining, A* Search, and a standard Backtracking baseline.

Key Findings:

- **Search vs. Inference:** Our experiments proved that search-based algorithms (Backtracking and A*) are far more effective for Futoshiki than purely logic-driven ones. While Forward Chaining was fast, its 30% success rate shows that deduction alone cannot solve complex puzzles.
- **Performance:** Backward Chaining, although reliable (100% success), suffered from extreme overhead as the grid size increased. In contrast, our A* solver, using the MRV heuristic, provided a balanced performance by intelligently pruning the search tree **ref1**.
- **Optimization:** The "Secret Solver" highlighted that combining search with aggressive constraint propagation (filtering illegal values before searching) is the most efficient way to solve 9×9 puzzles.

Future Work:

If we were to expand this project further, we would focus on:

- **Advanced Heuristics:** Implementing the AC-3 (Arc-Consistency) algorithm to prune domains more effectively before the A* search starts.
- **Solver Integration:** Using a professional SAT solver (like Z3 or PySAT) to compare the speed of industrial-grade logic engines against our custom implementations.
- **UI Improvements:** Enhancing the Flet GUI to allow users to input their own puzzles manually rather than just loading text files.

References

Puzzle Futoshiki. (2026). *Futoshiki puzzle rules*. https://www.puzzle-futoshiki.com/
Russell, S. J., & Norvig, P. (2020). *Artificial intelligence: A modern approach* (4th). Pearson.